

LONDON HOUSING AVM

Automated Valuation Models for London Residential Real Estate (1995–2024)

Trees vs. Neural Networks under Temporal Shift

418,201 transactions · 29 years · 4 model families

Why this project? London prices doubled relative to income over 29 years — can a model predict what a property is worth today, trained only on the past?

Neural Networks Course Project · FABIZ-ASE · 2026

The Problem & Why It Matters

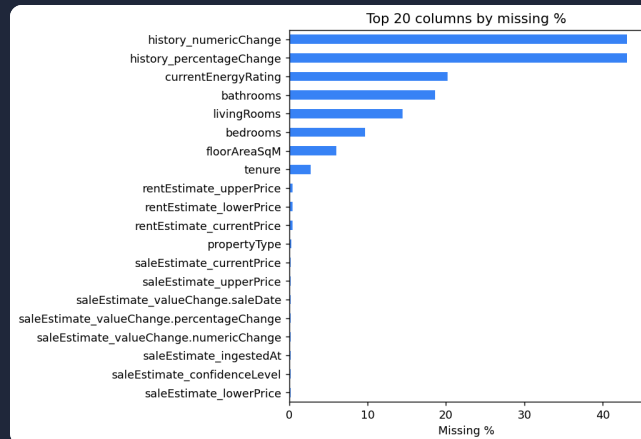
Motivation

- London — the most volatile residential market in the UK, with direct macro-economic impact
- Price-to-income: $\sim 4\times$ (1995) $\rightarrow >10\times$ (2006) $\rightarrow$ GFC $\rightarrow$ COVID "space race" $\rightarrow$ high rates 2022–2024
- AVMs (Automated Valuation Models) = scalable, objective, real-time valuation
- Central challenge: data leakage — `saleEstimate_*` / `rentEstimate_*` columns excluded from training (policy decision)
 - i These columns contain the vendor's own price estimate — using them as features would let the model see the answer. Excluded by policy.
- Target: `history_price` (continuous GBP, heavy-tailed log-normal distribution)

$\rightarrow$ Next: what does the data actually look like, and how do we split it fairly?

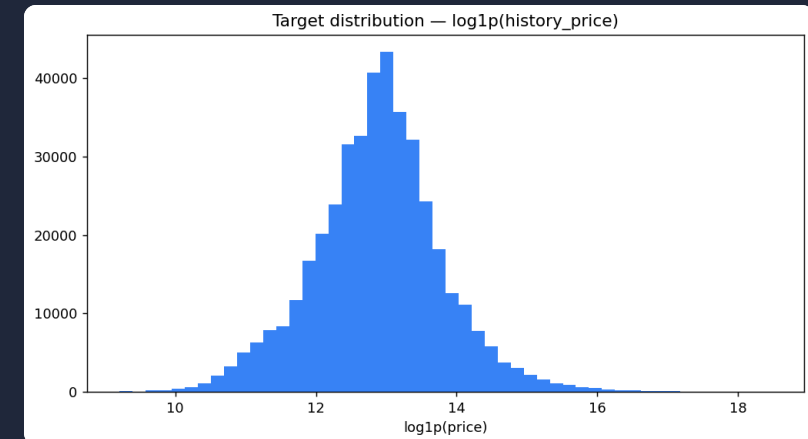
Dataset Quality — EDA Overview

Missingness profile and target distribution before any modeling



Top 20 features by missing % — vendor estimate columns dominate

i Vendor columns (`saleEstimate_*`, `rentEstimate_*`) are the most missing — and also excluded from mainline features. Both reasons point to the same decision.



`log1p(history_price)` distribution — near-Gaussian after transform

i `log1p(price)` is near-Gaussian → justifies PyTorch's log-target transformation on Slide 12.

→ Next: how we split 418k rows so the model never sees the future.

Dataset & Temporal Split

Calendar-elapsed split, not row-count

SET	PERIOD	ROWS (APPROX)
Train	1995-01-02 → 2015-03-04 (70% calendar)	~185k
Validation	2015-03-04 → 2018-04-13 (10% calendar)	~53k
Test	2018-04-13 → 2024-09-27 (20% calendar)	~130k

418,201 raw rows → 102,527 exact duplicates removed → quantile clipping (1%–99%) selected on validation

All preprocessing (imputation, scaling, OHE) fit exclusively on train — applied blind to val/test.

i Splitting by calendar time (not row index) prevents recent years — which have far more transactions — from leaking into the training window.

i If the scaler saw test data, it would know future price ranges. This is a form of data leakage.

→ Next: which model families competed on this split?

Model Families Compared

Same preprocessor · same split · same evaluation matrix

FAMILY	ARCHITECTURE
Baseline	NaiveMedian (zero-intelligence reference)
<i>i</i> NaiveMedian always predicts the training median (~£270k). It is the zero-intelligence floor — any real model must beat it.	
Linear	Ridge, ElasticNet
Tree Ensemble	HistGradientBoostingRegressor, RandomForestRegressor
NN sklearn	MLPRegressor (small / medium / large capacity scan)
NN PyTorch	MLPNet custom (dropout 0.2, AdamW, SmoothL1, log-target + bias init)
<i>i</i> Course-required model: explicit training loop, architecture diagram, and loss curves on Slides 12–13.	

Same evaluation matrix across all models — absolute fairness

→ Next: who won on the held-out test period 2018–2024?

Main Results — Static Holdout (Test 2018–2024)

The impact slide

MODEL	TEST RMSE (£)	TEST MAE (£)	R ²
HistGBR ✓	393,489	216,083	0.720
RandomForest	396,947	215,149	~0.715
Ridge	562,968	—	—
MLP_medium (sklearn)	645,104	—	—
TorchMLP (PyTorch)	729,943	—	—
NaiveMedian	927,646	561,899	—

HistGBR: 57.6% RMSE reduction vs NaiveMedian

5-QUANTILE BIN CLASSIFICATION (NON-TECHNICAL PROXY)

Price-bin Accuracy

Macro F1

49.51%

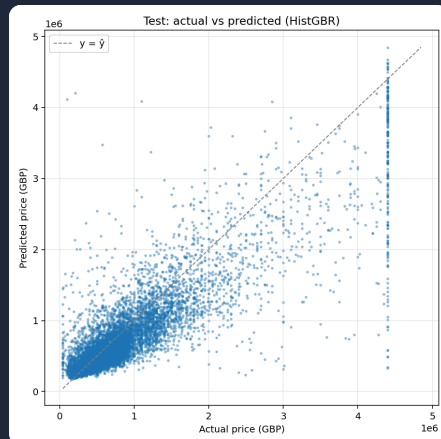
0.495

📌 R² = 0.720 means the model explains 72% of the variance in London house prices on unseen data from 2018–2024.

📌 Accuracy of 49.51% on 5 price bands ≈ almost 2× a random classifier (20%) — useful for stakeholders who think in bands, not £ errors.

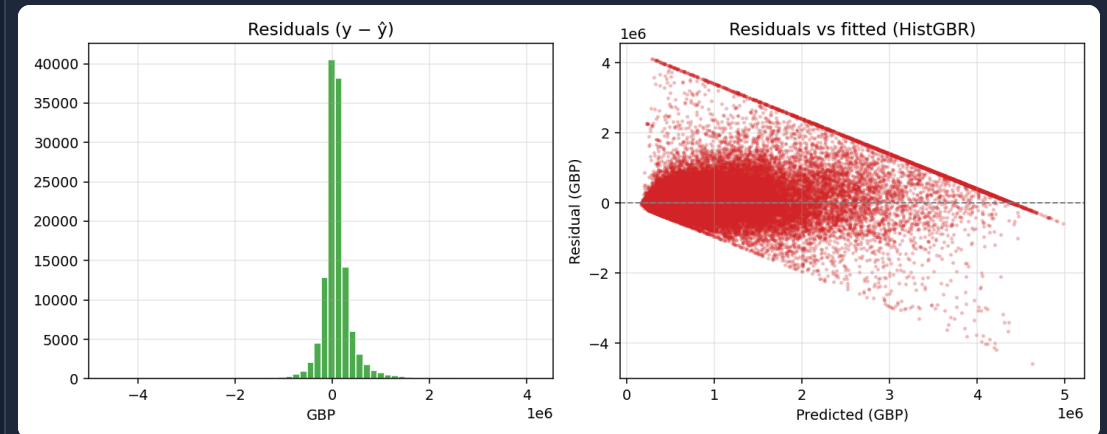
Regression Diagnostics — HistGBR Test Set

Where the winning model is accurate, and where it breaks



Predicted vs Actual · $R^2 = 0.720$ · Test 2018–2024

- Points on the dashed diagonal = perfect prediction. The spread above £2M shows where the model underestimates luxury properties.



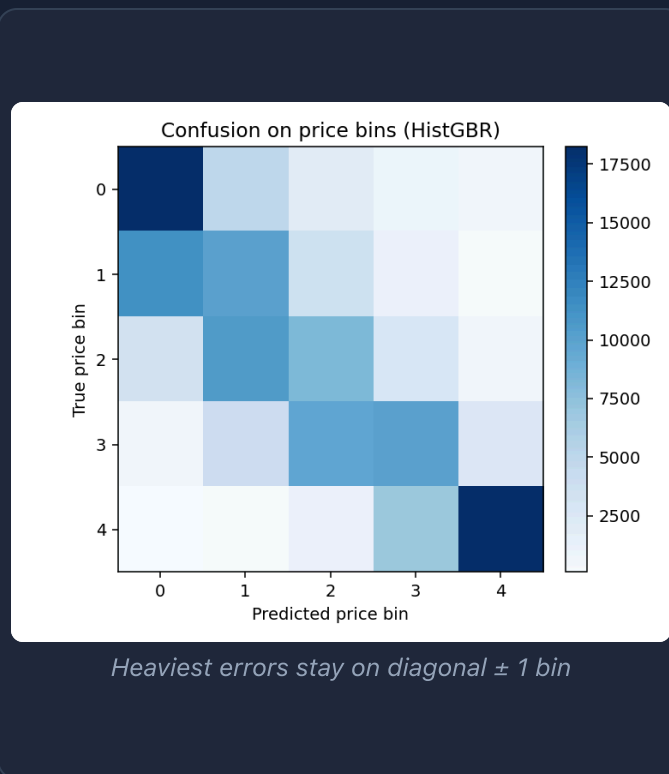
Residuals vs Predicted · heavy tail visible above £1.5M

- Residuals cluster tightly near zero but fan out above £1.5M — the heavy tail the model cannot fully capture. This drives the segment gate FAIL on Detached and Semi-Detached.

5-Bin Classification View — Non-Technical Accuracy Proxy

Continuous regression translated into discrete price bands for stakeholders

i Each row = true price band (cheapest=0, most expensive=4). Each column = predicted band. Diagonal = correct. Off-diagonal errors are mostly ± 1 bin — the model is rarely wrong by more than one price tier.

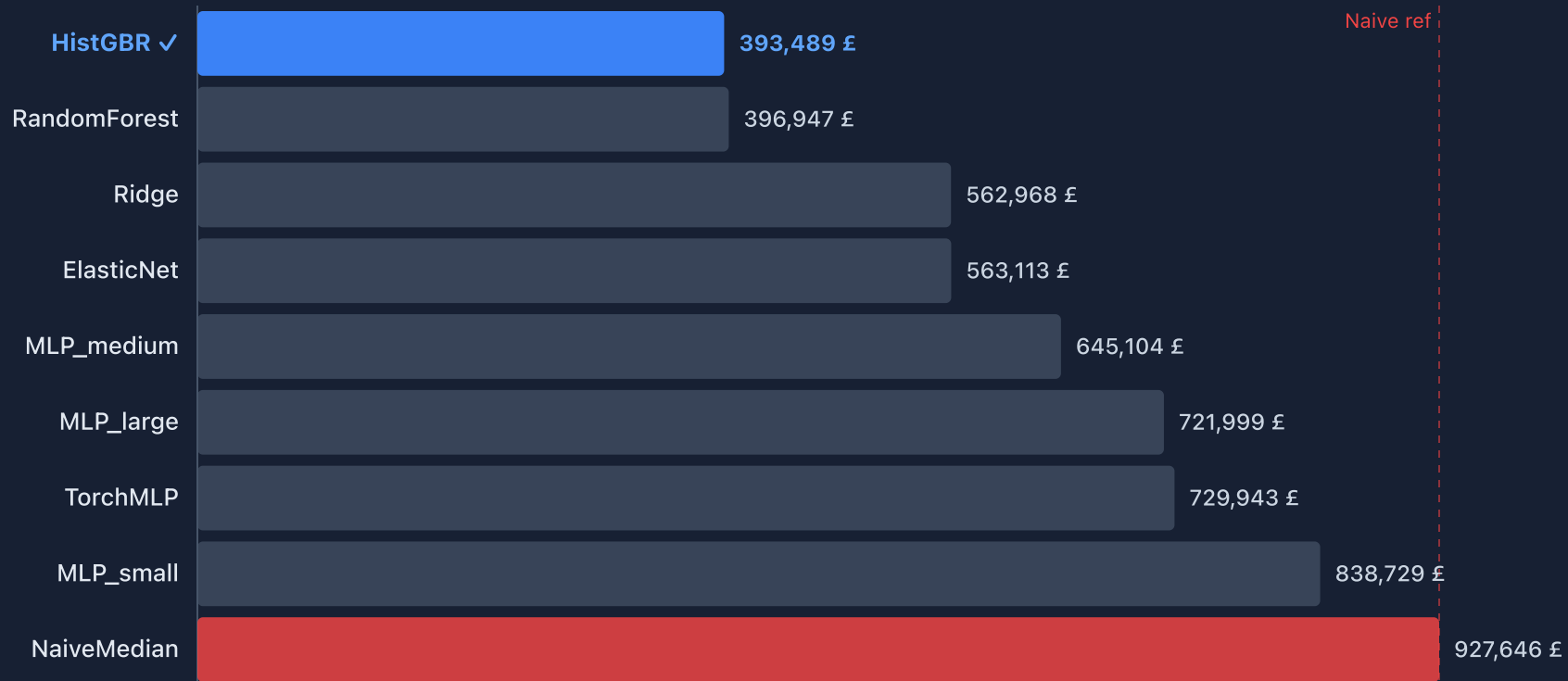


ACCURACY
49.51%

MACRO F1
0.495

RMSE Comparison — All Models

Test set 2018–2024 · sorted ascending · NaiveMedian = zero-intelligence baseline



Test RMSE (£) — sorted ascending

HistGBR wins by a thin margin over RandomForest; all NN variants sit above linear models on this static holdout.

i Only £3,458 RMSE difference between HistGBR and RandomForest — essentially tied on the static holdout. The walk-forward tells a different story.

i All NN variants sit 60–120% above the tree winners on this single holdout. Calendar shift (train on 1995–2015, test on 2018–2024 at much higher prices) disadvantages NNs more than boosting.

Why Trees Win on Static Holdout

Literature confirms our findings — Grinsztajn et al. 2022 · Shwartz-Ziv & Armon 2022

- Robustness to uninformative features — tree splits ignore noise columns; NNs do not

i Our dataset has many sparse OHE columns and several near-constant features. Trees ignore them; NNs propagate noise through all layers.

- Discontinuous boundaries — property prices have hard edges (postcode, catchment areas, taxes) → NN spectral bias hurts

i A postcode boundary can cause a £200k price jump in 100 metres. Trees model this as a split; NNs approximate it with smooth activation functions.

- No scaling requirements — trees are invariant to monotone transforms; NNs need precise preprocessing

On a single-shot holdout with large calendar shift (1995→2024), boosting clearly beats NNs

→ But wait — a single holdout is not the whole story. What happens when we test across multiple time windows?

Walk-Forward Validation — Where NNs Fight Back

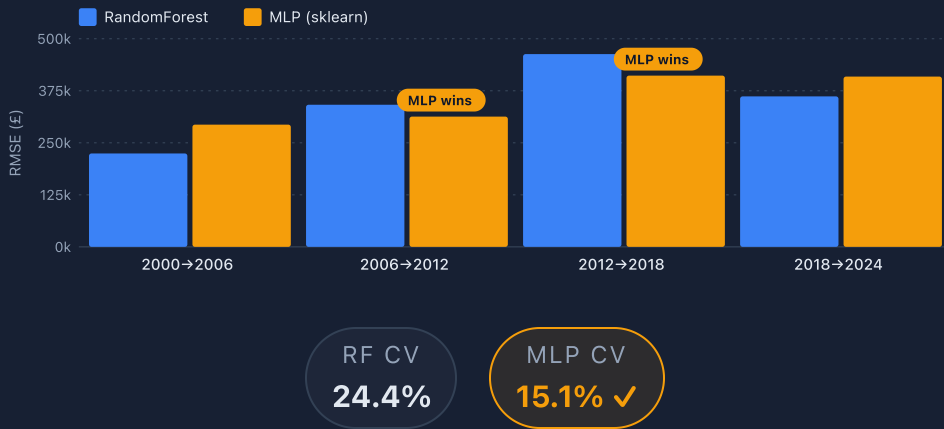
Protocol: 4 calendar folds, expanding window (2000→2006→2012→2018→2024)

METRIC	RANDOMFOREST	MLP (SKLEARN)
Mean RMSE	347,500 £	356,658 £
RMSE Std	84,775	53,909
CV%	24.4%	15.1% ✓

i CV% = RMSE standard deviation / mean RMSE. Lower = more consistent across different macro regimes. MLP CV is 38% lower than RF — it adapts better as the market shifts.

MLP beats RF in 2 of 4 folds · CV 38% lower → more temporally stable

In real production, where macro regimes shift, the NN is the more robust candidate.



PyTorch MLP — Architectural Safeguards

Three critical design choices for heavy-tailed financial data

- Log-target transformation — $\log_{10}(y) \rightarrow \text{near-Gaussian} \rightarrow \text{gradient stability}$

i Without this, the model trains on £300k–£5M values with high variance. After $\log_{10}$, the target range is ~12–15, much easier for gradient descent.

- Strategic bias initialization — `final_layer.bias = mean(log10(y_train))` → eliminates Initial Guessing Bias

i Without this, random initialisation can produce $\log(\text{price}) \approx 24$, meaning $\exp_{10}(24) \approx \text{£}26 \text{ billion}$ per property. Observed empirically on the first run.

- Log-clipping — `clamp(pred_log, min_log - margin, max_log + margin)` → impossible to predict £26B prices

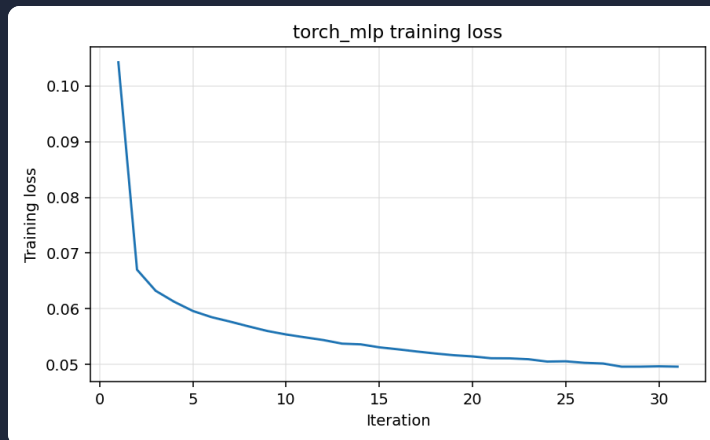
i Safety net: even if training diverges slightly, predictions stay within the observed price envelope $\pm$ a small margin.

- Architecture: `dropout(0.2)` · `AdamW` · `SmoothL1Loss` · early stopping on val RMSE in GBP space



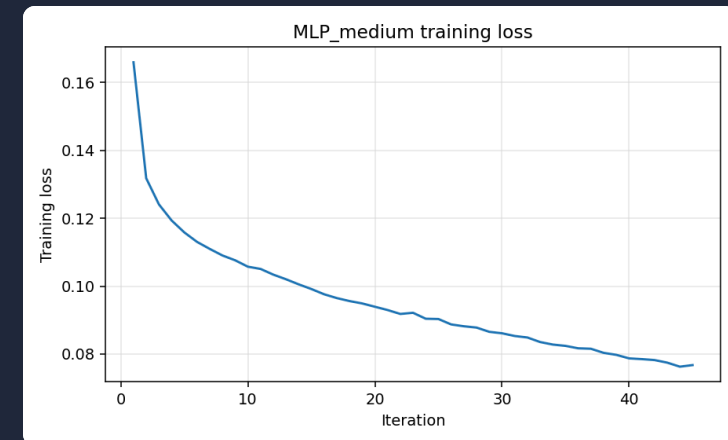
PyTorch MLP — Training Convergence

Pipeline-generated loss curves: PyTorch vs sklearn



TorchMLP · Best checkpoint epoch 21 · Val RMSE 428,185 £

- Val RMSE stops decreasing consistently after epoch 21 — early stopping restores that checkpoint. Training loss still falls, showing overfitting would occur without stopping.



sklearn MLP_medium · same split · Val RMSE plateau visible

- sklearn MLP uses StandardScaler on the target (not log), so early stopping is honest but the model cannot exploit the log-normal structure. This is why TorchMLP has a dedicated training loop.

Dual-Track & Honest Limitations

What works with vendor features vs. where the model fails

Assisted Track (with vendor features)

- AssistedHistGBR Test RMSE: 304,436 £ (vs 393,489 mainline)
- External benchmark saleEstimate_lowerPrice: 350,168 £
- But: depends on vendor data → not independently deployment-safe

i AssistedHistGBR RMSE (304,436 £) even beats the vendor's own estimate (350,168 £) — the model adds value on top of the vendor signal. But this requires vendor data at inference time.

Honest Limitations

- Segment gate FAIL on: Detached, Semi-Detached, price_band > 1M £
- MAPE = 33.8% → within-10% rate only 24.3% (below commercial AVM standard)
- Large calendar shift: train 1995–2015 vs test 2018–2024 (much higher prices)

i Semi-Detached RMSE = 825,628 £ vs overall 393,489 £ — more than 2× worse. These segments fail the automated release gate and would need targeted retraining before production.

Conclusions & Key Takeaways

i The full story in one sentence: a gradient boosting model trained on 20 years of London transactions predicts 2018–2024 prices with 57.6% lower error than a naive baseline, while neural networks — despite losing on this single holdout — prove more temporally stable across rolling windows.

RQ1 HistGBR dominates static holdout — R^2 0.720, 57.6% RMSE uplift vs naive

RQ2 MLP is 38% more stable on walk-forward (CV 15.1% vs 24.4%) → better production candidate under temporal drift

RQ3 Bias initialization + log-clipping are critical for PyTorch MLP convergence on heavy-tailed financial data

RQ4 5-bin classification proxy (accuracy 49.51%) translates regression into accessible language for non-technical stakeholders

⊕ No universal winner — trees for peak static accuracy, NNs for temporal stability in production

END OF PRESENTATION

Thank you

Questions?

London Housing AVM · FABIZ-ASE · 2026